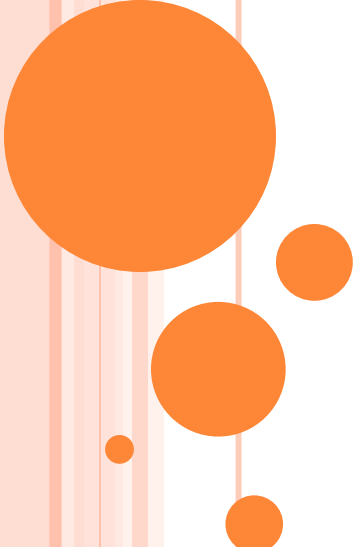
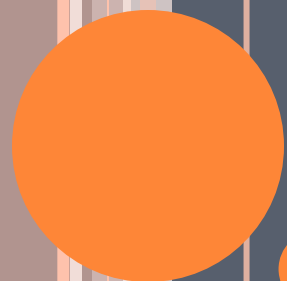




EXEMPLO TDD

Paulyne Jucá (paulyne@ufc.br)

Fonte: livro Test-Driven Development do Mauricio Aniche pela Casa do Código



LOJA VIRTUAL

LOJA VIRTUAL

- Imagine-se passeando em uma loja virtual qualquer na web. Ao selecionar um produto, o sistema coloca-o no seu carrinho de compras. Ao finalizar a compra, o sistema fala com a operadora de cartão de crédito, retira o produto do estoque, dispara um evento para que a equipe de logística separe os produtos comprados e te envia um e-mail confirmando a compra.
- O software que toma conta de tudo isso é complexo. Ele contém regras de negócio relacionadas ao carrinho de compras, ao pagamento, ao fechamento da compra.
- Mas, muito provavelmente, todo esse código não está implementado em apenas um único arquivo; esse sistema é composto por diversas pequenas classes, cada uma com sua tarefa específica.
- Desenvolvedores, quando pensam em teste de software, geralmente imaginam um teste que cobre o sistema como um todo. Um teste de unidade não se preocupa com todo o sistema; ele está interessado apenas em saber se uma pequena parte do sistema funciona.



LOJA VIRTUAL

- Um teste de unidade testa uma única unidade do nosso sistema. Geralmente, em sistemas orientados a objetos, essa unidade é a classe. Em nosso sistema de exemplo, muito provavelmente existem classes como “CarrinhoDeCompras”, “Pedido”, e assim por diante.
- A ideia é termos baterias de testes de unidade separadas para cada uma dessas classes; cada bateria preocupada apenas com a sua classe.



PROBLEMA

- Uma loja virtual precisa encontrar, dentro do seu carrinho de compras, os produtos de maior e menor valor.
- A gente isolaria o algoritmo que percorre o Carrinho de Compras e encontraria o maior e o menor



```
public class MaiorEMenor {

    private Produto menor;
    private Produto maior;

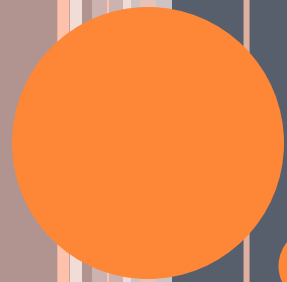
    public void encontra(CarrinhoDeCompras carrinho) {
        for(Produto produto : carrinho.getProdutos()) {
            if(menor == null || produto.getValor() < menor.getValor()) {
                menor = produto;
            }
            else if (maior == null ||
                produto.getValor() > maior.getValor()) {
                maior = produto;
            }
        }
    }

    public Produto getMenor() {
        return menor;
    }

    public Produto getMaior() {
        return maior;
    }

}
```





TESTE SEM JUnit

```
public class TestaMaiorEMenor {  
    public static void main(String[] args) {  
        CarrinhoDeCompras carrinho = new CarrinhoDeCompras();  
        carrinho.adiciona(new Produto("Liquidificador", 250.0));  
        carrinho.adiciona(new Produto("Geladeira", 450.0));  
        carrinho.adiciona(new Produto("Jogo de pratos", 70.0));  
  
        MaiorEMenor algoritmo = new MaiorEMenor();  
        algoritmo.encontra(carrinho);  
  
        System.out.println("O menor produto: " +  
            algoritmo.getMenor().getNome());  
        System.out.println("O maior produto: " +  
            algoritmo.getMaior().getNome());  
    }  
}
```





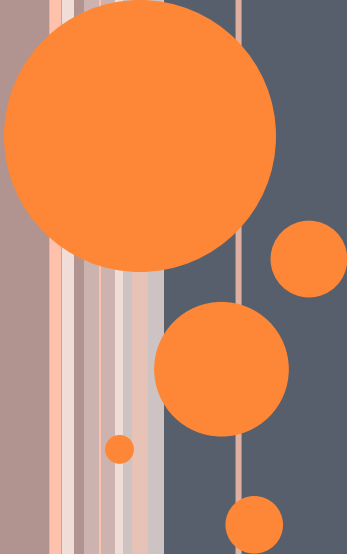
MAS SE A GENTE TROCAR A ORDEM
DOS ELEMENTOS NO CARRINHO?

```
public class TestaMaiorEMenor {  
    public static void main(String[] args) {  
        CarrinhoDeCompras carrinho = new CarrinhoDeCompras();  
        carrinho.adiciona(new Produto("Geladeira", 450.0));  
        carrinho.adiciona(new Produto("Liquidificador", 250.0));  
        carrinho.adiciona(new Produto("Jogo de pratos", 70.0));  
  
        MaiorEMenor algoritmo = new MaiorEMenor();  
        algoritmo.encontra(carrinho);  
  
        System.out.println("O menor produto: " +  
            algoritmo.getMenor().getNome());  
        System.out.println("O maior produto: " +  
            algoritmo.getMaior().getNome());  
    }  
}
```



0 menor produto: Jogo de pratos

Exception in thread "main" java.lang.NullPointerException
at TestaMaiorEMenor.main(TestaMaiorEMenor.java:12)



PQ?

```
public class MaiorEMenor {

    private Produto menor;
    private Produto maior;

    public void encontra(CarrinhoDeCompras carrinho) {
        for(Produto produto : carrinho.getProdutos()) {
            if(menor == null || produto.getValor() < menor.getValor()) {
                menor = produto;
            }
            else if (maior == null ||
                produto.getValor() > maior.getValor()) {
                maior = produto;
            }
        }
    }

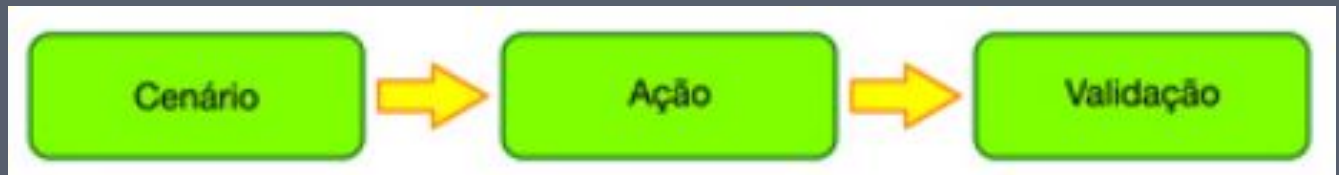
    public Produto getMenor() {
        return menor;
    }

    public Produto getMaior() {
        return maior;
    }

}
```



SE A GENTE NÃO TIVESSE TESTADO O
SEGUNDO CENÁRIO, DESCOBRIRÍAMOS
O BUG?



SOLUÇÃO? SISTEMATIZAR OS TESTES

CENÁRIOS POSSÍVEIS

- Produtos com valores em ordem crescente
- Produtos com valores em ordem decrescente
- Um único produto no carrinho



```
import org.junit.Assert;
import org.junit.Test;

public class TestaMaiorEMenor {

    @Test
    public void apenasUmProduto() {
        CarrinhoDeCompras carrinho = new CarrinhoDeCompras();
        carrinho.adiciona(new Produto("Geladeira", 450.0));

        MaiorEMenor algoritmo = new MaiorEMenor();
        algoritmo.encontra(carrinho);

        Assert.assertEquals("Geladeira",

            algoritmo.getMenor().getNome());
        Assert.assertEquals("Geladeira",
            algoritmo.getMaior().getNome());
    }
}
```

